

Chapter 18. Ingresses

[Chapter 17](#) delved into the purpose and creation of the Service primitive. Once there's a need to expose the application to external consumers, selecting an appropriate Service type becomes crucial. The most practical choice often involves creating a Service of type `LoadBalancer`. Such a Service offers load balancing capabilities by assigning an external IP address accessible to consumers outside the Kubernetes cluster.

However, opting for a `LoadBalancer` Service for each externally reachable application has drawbacks. In a cloud provider environment, each Service triggers the provisioning of an external load balancer, resulting in increased costs. Additionally, managing a collection of `LoadBalancer` Service objects can lead to administrative challenges, as a new object must be established for each externally accessible microservice.

To mitigate these issues, the Ingress primitive comes into play, offering a singular, load-balanced entry point to an application stack. An Ingress possesses the ability to route external HTTP(S) requests to one or more Services within the cluster based on an optional, DNS-resolvable host name and URL context path. This chapter will guide you through the creation and access of an Ingress.

COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objective:

- Know how to use Ingress controllers and Ingress resources

ACCESSING AN INGRESS IN MINIKUBE

Accessing an Ingress in minikube requires special handling. Refer to the Kubernetes tutorial [“Set up Ingress on Minikube with the NGINX Ingress Controller”](#) for detailed instructions.

Working with Ingresses

The Ingress exposes HTTP (and optionally HTTPS) routes to clients outside of the cluster through an externally-reachable URL. The routing rules configured with the Ingress determine *how* the traffic should be routed.

Cloud provider Kubernetes environments will often deploy an external load balancer. The Ingress receives a public IP address from the load balancer. You can configure rules for routing traffic to multiple Services based on specific URL context paths, as shown in [Figure 18-1](#).

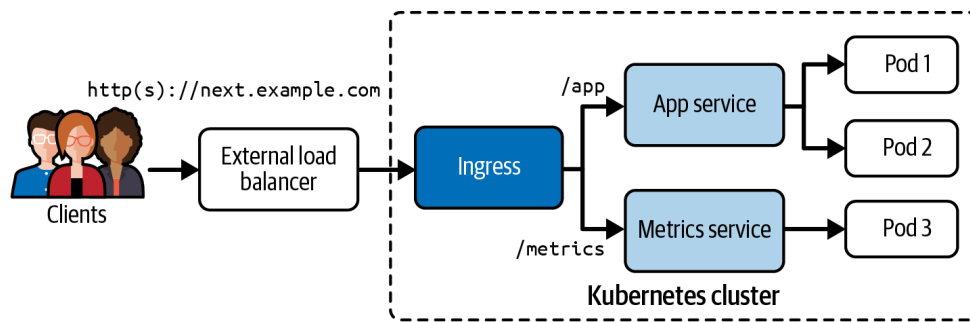


Figure 18-1. Managing external access to the Services via HTTP(S)

The scenario depicted in [Figure 18-1](#) instantiates an Ingress as the sole entry point for HTTP(S) calls to the domain name *next.example.com*. Based on the provided URL context, the Ingress directs the traffic to either of the fictional Services: one designed for a business application and the other for fetching metrics related to the application.

Specifically, the URL context path `/app` is routed to the App Service responsible for managing the business application. Conversely, sending a request to the URL context `/metrics` results in the call being forwarded to the Metrics Service, which is capable of returning relevant metrics.

Installing an Ingress Controller

For Ingress to function, an Ingress controller is essential. This controller assesses the set of rules outlined by an Ingress, dictating the routing of traffic. The choice of Ingress controller often depends on the specific use cases, requirements, and preferences of the Kubernetes cluster administrator. Noteworthy examples of production-grade Ingress controllers include the [F5 NGINX Ingress Controller](#) or the [AKS Application Gateway Ingress Controller](#). Additional options can be explored in the [Kubernetes documentation](#).

With the NGINX Ingress Controller we're using, you should find at least one Pod that runs the Ingress controller after installing it. This output renders the Pod created by the NGINX Ingress controller residing in the namespace `ingress-nginx`:

```
$ kubectl get pods -n ingress-nginx
```

NAME	READY	STATUS	RESTARTS
ingress-nginx-admission-create-qghrp	0/1	Completed	0
ingress-nginx-admission-patch-56z26	0/1	Completed	1
ingress-nginx-controller-7c6974c4d8-2gg8c	1/1	Running	0

Once the Ingress controller Pod transitions into the `Running` status, you can assume that the rules defined by Ingress objects will be evaluated.

Deploying Multiple Ingress Controllers

Certainly, deploying multiple Ingress controllers within a single cluster is a feasible option, especially if a cloud provider has preconfigured an Ingress controller in the Kubernetes cluster. The Ingress API introduces the attribute `spec.ingressClassName` to facilitate the selection of a specific controller implementation by name. To identify all installed Ingress classes, you can use the following command:

```
$ kubectl get ingressclasses
```

NAME	CONTROLLER	PARAMETERS	AGE
nginx	k8s.io/ingress-nginx	<none>	14m

Kubernetes determines the default Ingress class by scanning for the annotation `ingressclass.kubernetes.io/is-default-class: "true"` within all Ingress class objects. In scenarios where Ingress objects do not explicitly specify an Ingress class using the attribute `spec.ingressClassName`, they automatically default to the Ingress class marked as the default through this annotation. This mechanism provides flexibility in managing Ingress classes and allows for a default behavior when no specific class is specified in individual Ingress objects.

While Kubernetes doesn't prevent this configuration, when multiple Ingress classes have the `ingressclass.kubernetes.io/is-default-class: "true"` annotation assigned, it creates ambiguous behavior. Ingress resources without an explicit `ingressClassName` will typically fail to create, with validation errors about multiple defaults being found, as the system cannot deterministically choose which controller should handle them.

Configuring Ingress Rules

When creating an Ingress, you have the flexibility to define one or multiple rules. Each rule encompasses the specification of an optional host, a set of URL context paths, and the backend responsible for routing the incoming traffic. This structure allows for fine-grained control over how external HTTP(S) requests are directed within the Kubernetes cluster, catering to different services based on specified conditions. [Table 18-1](#) describes the three rules.

Table 18-1. Ingress rules

Type	Example	Description
An optional host	<code>next.example.com</code>	If provided, the rules apply to that host. If no host is defined, all inbound HTTP(S) traffic is handled (e.g., if made through the IP address of the Ingress).
A list of paths	<code>/app</code>	Incoming traffic must match the host and path to correctly forward the traffic to a Service.
The backend	<code>app-service:8080</code>	A combination of a Service name and port. The Ingress allows access to intra-cluster Services defined as type <code>ClusterIP</code> .

An Ingress controller can optionally define a default backend that is used as a fallback route should none of the configured Ingress rules match.

You can learn more about it in the [documentation of the Ingress primitive](#).

Creating Ingresses

You can create an Ingress with the imperative `create ingress` command. The main command-line option you need to provide is `--rule`, which defines the rules in a comma-separated fashion. The notation for each key-value pair is `<host>/<path>=<service>:<port>`. Let's create an Ingress object with two rules:

```
$ kubectl create ingress next-app \
  --rule="next.example.com/app=app-service:8080" \
  --rule="next.example.com/metrics=metrics-service:9090"
ingress.networking.k8s.io/next-app created
```

If you look at the output of the `create ingress --help` command, more fine-grained rules can be specified.

SUPPORT FOR TLS TERMINATION

Port 80 for HTTP traffic is implied, as we didn't specify a reference to a TLS Secret object. If you have specified `tls=mysecret` in the rule definition, then the port 443 would be listed here as well. For more information on enabling HTTPS traffic, see the [Kubernetes documentation](#). The exam does not cover configuring TLS termination for an Ingress.

Using a YAML manifest to define Ingress is often more intuitive and preferred by many. It provides a clearer and more structured way to express

the desired configuration. An Ingress defined as a YAML manifest is shown in [Example 18-1](#).

Example 18-1. An Ingress defined by a YAML manifest

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: next-app
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$1 ❶
spec:
  rules:
    - host: next.example.com ❷
      http:
        paths:
          - backend:
              service:
                name: app-service
                port:
                  number: 8080
              path: /app
              pathType: Exact
          - backend: ❸
              service:
                name: metrics-service
                port:
                  number: 9090
              path: /metrics
              pathType: Exact
```

- ❶ Assigns an NGINX Ingress-specific annotation for rewriting the URL
- ❷ Defines the rule that maps the `app-service` backend to the URL `next.example.com/app`
- ❸ Defines the rule that maps the `metrics-service` backend to the URL `next.example.com/metrics`

The Ingress YAML manifest contains one major difference from the live object representation created by the imperative command: the assignment of an Ingress controller annotation. Some Ingress controller implementations provide annotations to customize their behavior. You can find the full list of annotations that come with the NGINX Ingress controller in the [corresponding documentation](#).

Defining Path Types

The previous YAML manifest demonstrates one of the options for specifying a path type via the attribute `spec.rules[ ].http.paths[ ].pathType`. The path type defines how an incoming request is evaluated against the declared path. [Table 18-2](#) indicates the evaluation for incoming requests and their paths. See the [Kubernetes documentation](#) for a more comprehensive list.

Table 18-2. Ingress path types

Path type	Rule	Incoming request
Exact	/app	Matches /app but does not match /app/test or /app/
Prefix	/app	Matches /app, /app/, and /application but does not match /admin

The key distinction between the `Exact` and `Prefix` path types lies in their treatment of trailing slashes. The `Prefix` path type focuses solely on the provided prefix of a URL context path, allowing it to accommodate requests with URLs that include a trailing slash. In contrast, the `Exact` path type is more stringent, requiring an exact match of the specified URL context path without considering a trailing slash.

Listing Ingresses

Listing Ingresses can be achieved with the `get ingress` command. You will see some of the information you specified when creating the Ingress (e.g., the hosts):

```
$ kubectl get ingress
```

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
next-app	nginx	next.example.com	192.168.66.4	80	5m38s

The Ingress automatically selected the default Ingress class `nginx` configured by the Ingress controller. You can find that information under the `CLASS` column. The value listed under the `ADDRESS` columns is the IP address provided by the external load balancer.

Rendering Ingress Details

The `describe ingress` command is a valuable tool for obtaining detailed information about an Ingress resource. It presents the rules in a clear table format, which aids in understanding the routing configura-

tions. Additionally, when troubleshooting, it's essential to pay attention to any additional messages or events.

In the provided output, it's evident that there might be an issue with the Services named `app-service` and `metrics-service` that are mapped in the Ingress rules. This discrepancy between the specified services and their existence can lead to routing errors:

```
$ kubectl describe ingress next-app
Name:                next-app
Labels:              <none>
Namespace:           default
Address:             192.168.66.4
Ingress Class:       nginx
Default backend:     <default>
Rules:
  Host                Path    Backends
  ----                -
  next.example.com    /app      app-service:8080 (<error: endpoints \
                    /metrics  metrics-service:9090 (<error: endpoints \
                    "app-service" not found>)
                    "metrics-service" not found>)
Annotations:         <none>
Events:
  Type    Reason    Age                From                ...
  ----    -
  Normal  Sync      6m45s (x2 over 7m3s)  nginx-ingress-controller ...
```

Furthermore, observing the event log that shows syncing activity by the Ingress controller is crucial. Any warnings or errors in this log can provide insights into potential issues during the synchronization process.

To address the problem, ensure that the specified Services in the Ingress rules actually exist and are accessible within the Kubernetes cluster. Additionally, review the event log for any relevant messages that might indicate the cause of the discrepancy.

Let's resolve the issue of not being able to route to the backends configured in the Ingress object. The following commands create the Pods and Services:

```
$ kubectl run app --image=k8s.gcr.io/echoserver:1.10 --port=8080 \
-l app=app-service
pod/app created
$ kubectl run metrics --image=k8s.gcr.io/echoserver:1.10 --port=8080 \
-l app=metrics-service
pod/metrics created
$ kubectl create service clusterip app-service --tcp=8080:8080
```

```

service/app-service created
$ kubectl create service clusterip metrics-service --tcp=9090:8080
service/metrics-service created

```

Inspecting the Ingress object doesn't show any errors for the configured rules. If you're now able to see a list of resolvable backends along with the corresponding Pod virtual IP addresses and ports, the Ingress object is correctly configured, and the backends are recognized and accessible:

```

$ kubectl describe ingress next-app
Name:                next-app
Labels:              <none>
Namespace:           default
Address:             192.168.66.4
Ingress Class:       nginx
Default backend:     <default>
Rules:
  Host                Path    Backends
  ----                -
  next.example.com    /app    app-service:8080 (10.244.0.6:8080)
                     /metrics metrics-service:9090 (10.244.0.7:8080)
Annotations:         <none>
Events:
  Type    Reason    Age                From                Message
  ----    -
  Normal  Sync      13m (x2 over 13m)  nginx-ingress-controller  Scheduled for

```

It's worth coming back to the Ingress details if you experience any issues with routing traffic through an Ingress endpoint.

Accessing an Ingress

To enable the routing of incoming HTTP(S) traffic through the Ingress and subsequently to the configured Service, it's crucial to set up a DNS entry mapping to the external address. This typically involves configuring either an Address record (A record) or a Canonical Name (CNAME) record. The [ExternalDNS project](#) is a valuable tool that can assist in managing these DNS records automatically.

For local testing on a Kubernetes cluster on your machine, follow these steps:

1. Find the IP address of the load balancer used by the Ingress.
2. Add the IP address to hostname mapping to your */etc/hosts* file.

By adding the IP address to your local */etc/hosts* file, you will resolve the domain name locally, allowing you to test the behavior of the Ingress

without relying on external DNS records:

```
$ kubectl get ingress next-app \
  --output=jsonpath="{.status.loadBalancer.ingress[0]['ip']}"
192.168.66.4
$ sudo vim /etc/hosts
...
192.168.66.4    next.example.com
```

You can now send HTTP requests to the backend. This call matches the `Exact` path rule and therefore returns a HTTP 200 response code from the application:

```
$ wget next.example.com/app --timeout=5 --tries=1
--2021-11-30 19:34:57--  http://next.example.com/app
Resolving next.example.com (next.example.com)... 192.168.66.4
Connecting to next.example.com (next.example.com)|192.168.66.4|:80... \
connected.
HTTP request sent, awaiting response... 200 OK
```

This next call uses a URL with a trailing slash. The Ingress path rule doesn't support this case, and therefore the call doesn't go through. You will receive an HTTP 404 response code. For the second call to work, you'd have to change the path rule to `Prefix`:

```
$ wget next.example.com/app/ --timeout=5 --tries=1
--2021-11-30 15:36:26--  http://next.example.com/app/
Resolving next.example.com (next.example.com)... 192.168.66.4
Connecting to next.example.com (next.example.com)|192.168.66.4|:80... \
connected.
HTTP request sent, awaiting response... 404 Not Found
2021-11-30 15:36:26 ERROR 404: Not Found.
```

You can observe the same behavior for the Metrics Service configured with the URL context path `metrics`. Feel free to try that out as well.

Summary

The resource type Ingress defines rules for routing cluster-external HTTP(S) traffic to one or many Services. Each rule defines a URL context path to target a Service. For an Ingress to work, you first need to install an Ingress controller. An Ingress controller periodically evaluates those rules and ensures that they apply to the cluster. To expose the Ingress, a cloud provider usually stands up an external load balancer that lends an external IP address to the Ingress.

Exam Essentials

Know the difference between a Service and an Ingress

An Ingress is not to be confused with a Service. The Ingress is meant for routing cluster-external HTTP(S) traffic to one or many Services based on an optional hostname and mandatory path. A Service routes traffic to a set of Pods.

Understand the role of an Ingress controller

An Ingress controller needs to be installed before an Ingress can function properly. Without installing an Ingress controller, Ingress rules will have no effect. You can choose from a range of Ingress controller implementations, all documented on the Kubernetes documentation page. Assume that an Ingress controller will be pre-installed for you in the exam environment.

Practice the definition of Ingress rules

You can define one or many rules in an Ingress. Every rule consists of an optional host, the URL context path, and the Service DNS name and port. Try defining more than a single rule and how to access the endpoint. You will not have to understand the process for configuring TLS termination for an Ingress—this aspect is covered by the CKS exam.

Sample Exercises

Solutions to these exercises are available in [Appendix A](#).

1. Your company has two microservices running in the same Kubernetes cluster: A frontend application that serves the main website, and an API backend service that handles API requests. Both services need to be accessible from outside the cluster through the same domain name (*app.example.com*) but with different URL paths. Use the [F5 NGINX Ingress controller](#).

Create a namespace called `webapp`. Deploy two applications: a front-end application (use `nginx:1.29.1-alpine` image, name: `frontend`, two replicas), and an API application (use the `httpd:2.4.65-alpine` image, name: `api`, two replicas).

Create `ClusterIP` Services for both Deployments, a Frontend Service on port 80, an API Service on port 80.

Create an Ingress resource that uses the host *app.example.com* and routes the paths `/` and `/app` to the Frontend Service. It also routes

/api to the API Service. Use the `nginx` Ingress class. Verify the proper Ingress configuration by making a call to it.

2. Your team wants to implement a blue-green deployment strategy using Ingress. You need to create an Ingress configuration that can switch traffic between two versions of an application, with the ability to do canary testing before full switchover. The team decided to use the [F5 NGINX Ingress controller](#). You may reference the controller-specific annotations from the [documentation](#).

Create a namespace called `production-apps`. Deploy two versions of an application: a blue version (current) named `app-blue` with three replicas with the container image `nginxdemos/hello:0.3-plain-text`, and a green version (new) named `app-green` with three replicas with the container image `nginxdemos/hello:0.4-plain-text`. Create two Ingress resources in the `production-apps` namespace. A main ingress routing to blue version, and a canary ingress for testing green version (20% traffic). Assign the hostname

`app.production.com`. The traffic routing should look as follows: Default traffic goes to blue version. Twenty percent of traffic goes to green version (canary).

Add an entry in `/etc/hosts` that maps the load balancer IP address to the host `app.production.com`. Verify access to the Ingresses via this hostname. Test the blue-green traffic routing distribution.